

Code Translation

Ashton Schutzman - aschutzm@andrew.cmu.edu

Joaquin Wang Zhang - joaquinw@andrew.cmu.edu

Leo Hsu - lhsu2@andrew.cmu.edu

Introduction:

Recent advances in multimodal generative AI have held out the promise of new directions for gap-filling between visual and text modalities. In this paper, we explore one such intersection: cross-language translation from source code image representations in a programming language to a functional version of the code in a target language. Precisely, our system takes in an image or screenshot code of a source programming language (e.g., Java) as input and compiles it into a target language (e.g., C#), enabling easy program paradigm shifts, facilitating learning, cross-platform development, and improving productivity. First, the system needs to correctly decipher a code image to clean and adequately structure the text. Secondly, it must correctly translate the parsed source code to the correct code in the target language semantically. By combining these two parts, we expect to have a working prototype that can accurately translate code from images to functional code. We look to improve the model's performance on code translation through parameter-efficient fine-tuning methodologies. We aim to match the translation performance of codet5-base with full fine-tuning, but instead by fine-tuning only a fraction of the parameters using LoRA and Top-K layer tuning. One motivation is for users to be able to extract code from written media and use the translated code in their corresponding field.

Datasets & Tasks:

Regarding our task, we aim to improve the performance of the pre-trained CodeT5 model for code translation through parameter-efficient fine-tuning. Particularly in the CodeT5 research paper, the codet5-base model does impressively for Java to C# translation, achieving a BLEU score of 84.03. This is because they used **full** fine-tuning for the model. We strive to take the pretrained model, which only “understands” code and its structure, but does not do well with translation tasks, and achieve good performance through LoRA and Top K layers. Regarding our chosen datasets, the image-to-code structure would be fed a dataset comprising 200 GitHub screenshots of LeetCode solutions in Python and Java to detect and transcribe. We will be using the Normalized Levenshtein Distance (NLD) metric to determine how well our image-to-code submodel is performing. For the code-to-code structure, we will use the code_x_glue dataset to fine-tune our model as well as evaluate the performance of our translations. The dataset contains code translations between Java and C#, and is broken down into 10,300 training, 500 validation, and 1000 testing instances. We evaluate the model's output using Exact Match Accuracy (EM) to measure whether the predicted solution exactly matches the target code. Additionally, we use the

BLEU score to assess the degree of similarity between the generated output and the reference implementation, treating the target code as the ground-truth label.

Related Works:

A significant component of our project is our model's capability to transcribe our code screenshot inputs into the same source language before translating them into our target language. Bao et al. (2020) leveraged a CNN to remove non-code or noisy frames of a programming screencast before establishing a code editor subsection within the frame. Finally, the authors utilized the Google Vision API and Tesseract optical character recognition (OCR) engines to extract source code before implementing corrections with the help of a large source code corpus. Khormi et al. (2020) evaluated six different OCR engines for accuracy. The investigation concluded that the Google Drive API and ABBYY scored the highest when assessed on the Normalized Levenshtein Distance (NLD) metric, scoring around 96 and 94 percent. Other findings include that Java was the easiest to extract, followed by C# and Python in that order, as well as how much font and font sizing can affect performance, especially for lower-performing engines. Finally, Python's whitespace reduced accuracy for most engines, with an engine that included trimming increased performance by about 6 percent.

Traditional transcompilers in the past have heavily relied on a rule-based system, effectively tokenizing an input source code and converting it into an Abstract Syntax Tree (AST) for rewriting, but limiting performance due to its dependence on language-specific knowledge. Roziere et al. (2020) alleviated this with an unsupervised approach using cross-lingual language model pretraining (XLM), denoising auto-encoding (DAE), and back-translation. Trained only on monolingual Github code in C++, Java, and Python, their shared sequence-to-sequence encoder-decoder model outperformed other rule-based and commercial systems on GeeksforGeeks parallel functions and unit test benchmarks. Similarly, Liu et al. (2023) proposed their model, SDA-Trans, which enhanced a transformer-based encoder-decoder model with syntax-aware encoding via Graph Attention Networks (GATs), domain-adversarial training for language-invariant feature extraction, and a multi-stage training pipeline. Trained on CodeSearchNet (250k Python, 170k Java functions), SDA-Trans matched TransCoder on Python-Java translations but outperformed and had more exact matches for the Python-C++ bidirectional translation. This emphasizes the importance of syntax and domain features for SDA-Trans's success and its generalization ability to unseen languages, even with limited data.

The main model we are fine-tuning with LoRA and Top-K is codet5-base, an encoder-decoder Transformer. Introduced by Wang, Yue, et al. (2021), the pre-trained model is designed for both code understanding and generation. It uses identifier-aware denoising pre-training, which helps the model recognize and recover masked identifiers, and incorporates bimodal dual generation to align natural language (NL) with programming language (PL),

enabling it to generate one from the other. With full fine-tuning, CodeT5 achieves strong performance on code translation tasks, scoring 84.03 on BLEU and 65.90% EM for Java to C#. Similarly, Ahmad, Wasi Uddin, et al. (2023) introduced the AVATAR dataset, a supervised Java-Python code translation corpus. They fine-tuned CodeT5 on this dataset, achieving a BLEU score of 62.7.

Traditional fine-tuning of large pretrained models like BERT involves updating all of the model's parameters for each downstream task, proving to be inefficient for low-resource tasks, multi-task learning, and deployment on constrained devices. To address this, Houlsby et al. (2019) introduced adapter-based fine-tuning, where only small, trainable adapters are inserted between layers and trained during fine-tuning. Their work motivated the development of Top-K layer fine-tuning, where only the top K layers are fine-tuned while the rest are kept frozen, often achieving comparable results to full fine-tuning with far less computational cost. Similarly, Low-Rank Adaptation (LoRA), introduced by Hu et al. (2021), offers another low-parameter fine-tuning strategy by freezing pre-trained weights and injecting trainable low-rank matrices to specific Transformer layers. This reduces the number of trainable parameters while preserving model quality. LoRA has shown performance comparable to, or even outperforming, full fine-tuning models in various NLP tasks, making it an efficient choice for adapting CodeT5 to our translation task.

Methods:

Our project consists of a two-stage multimodal pipeline: extracting source code from images and translating the extracted code from one programming language to another. We designed this architecture to isolate the challenges of visual code understanding from the transformation of programming languages, allowing us to evaluate each independently. In the first stage, we aim to convert an image containing a source code picture into clean, structured text. For this, we will use the OCR capabilities in the Google Vision API as our baseline. To improve output quality, we will compare pre-processing methods like OpenCV's binarize functionality versus Pillow's image enhancement on a Normalized Levenshtein Distance (NLD) metric. The second stage is to use the CodeT5 model to translate source code from one language to another. The pre-trained CodeT5 model's performance without fine-tuning will act as our baseline, and we'll compare its performance after we fine-tune using two parameter-efficient methods, Top-K layer and LoRA. Both these approaches aim to achieve high-quality translations using a fraction of the number of trainable parameters. By comparing the effectiveness of these methods on standard metrics such as BLEU and Exact Match, we aim to observe how closely parameter-efficient fine-tuning can match or exceed the performance of full fine-tuning.

Experiments:

Our first of three experiments, “Image to Code,” evaluated our application of the Google Vision API on approximately 200 screenshots, 100 each of Python and Java. Despite variations in IDE appearances, text color, and font sizes across the dataset, the baseline results showed strong performance, with NLD of 0.039 and 0.0364 for Python and Java, shown respectively in Appendix (i). To further improve readability, we implemented a universal text normalization strategy on top of two preprocessing techniques using OpenCV’s binarization and Pillow’s ImageEnhance modules. From the generated NLD scores, we observed that binarizing the images increased recognition difficulty, whereas enhancing the image quality beat the baseline consistently. Appendices (ii) - (iii) highlight the improvement achieved through enhancement. To provide a clearer picture of accuracy, the enhanced Python set included 77 samples with a score below 0.05 and 92 within 0.10, while the enhanced Java set had 78 and 94 within those respective thresholds. Overall, it takes roughly 30 seconds to run all preprocessing methods and evaluation across 100 language-specific screenshots on Google Colab’s T4 GPU.

The second experiment is the “Code Translation,” where we used the pre-trained CodeT5 models for both the base model, which has 220 million parameters, and the small model, which has 60 million parameters. For a baseline understanding, we first evaluated how well these models performed without any fine-tuning. Using BLEU score and EM as performance metrics, a test dataset of 1000 translated codes from code_x_glue, the highest BLEU score achieved is approximately 0.038, notably from codet5-base’s C# to Java translation. On the other hand, translating from Java to C# with the same model yielded lower BLEU performance. Furthermore, we saw that the small model performed worse on the BLEU metric, while all of them got 0.0% exact match, as shown in Appendix (iv). Each baseline test takes roughly 4 to 5 minutes to test with T4.

We then fine-tuned the CodeT5 model with LoRa using a rank = 8, and to our surprise, fine-tuning it using 10300 training examples achieved a BLUE score of 0.9064, which outperforms the full-finetuning score of 0.8403 mentioned in the paper for Java to C#. Our scores for EM performance are slightly worse, as shown in Appendix (v). The same pattern holds for C# to Java. Using Colab’s T4 GPU, LoRA tasks have a 35-minute training runtime and 25 minutes for testing/evaluation. Regarding Top-K fine-tuning, we fine-tuned by varying the number of top layers and learning rate for codet5-base. As expected, we noticed that an increase in K resulted in lower fine-tuning training loss and a higher BLEU score and EM percentage. Appendix (vi) illustrates the training loss per training epoch as K varies, while Appendix (vii) illustrates the performance as learning rate decreases from the default setting. The Top-K tasks take around 40 minutes to train and 20 minutes to test/evaluate on T4, and as we lower the K, the compilation time reduces as well..

Finally, when we combined our two submodels together for the final pipeline, we achieved a worse performance with a BLEU score and a significant decrease in Exact Match for both Java to C# translations and vice versa, as shown in Appendix (ix). This is to be expected as

any recognition errors in the first submodel will be compounded, only increasing the difficulty for the translation model. This is perfectly illustrated with the Exact Match metric, as any recognition deviation heavily punishes that accuracy score and will cause translation errors. Despite this, our BLEU score remains in the range of 0.67 to 0.75, producing above-average results in image-to-code translations. Ultimately, it shows that our model is still decently effective in producing translations when given a screenshot of a different language. The entire pipeline takes T4 around an hour to generate code strings with the OCR model, as well as to train, evaluate, test, and output translations from the codeT5 model.

Code Overview:

Regarding our character recognition submodel, we wrote the code shown in Appendix (x)-(xiii) to call on the Google Vision API, both preprocessing methods (Pillow's ImageEnhance and CV2's Binarize), NLD metric determination, and the overall run function calls.

For the LoRA fine-tuning shows in Appendix (xiv) - (xv), very similar to the homework implementation, we first defined LoRALinear, which is a subclass of nn.Linear but adds two small trainable matrices of lora_A and lora_B, and a dropout layer. In the forward, it always does the base linear transformation, and if the weights are not merged, it adds the scaled lora_B * lora_A. The train either merges the LoRA weights so that they receive gradient updates or subtracts these weights for training. Now, the replace_with_lora function goes through every submodel and whenever it finds a T5Attention, it gets the keys and queries, creates a new LoRALinear of the same shape, copies over the weights, and sets the attention to use these weights. The freeze_non_lora function iterates through the parameters in the model, and if it is not a LoRA parameter, freezes it. Next, the encode function encodes the code examples from our data with the codet5-base tokenizer. We then proceed to train/fine-tune this model by calling the functions above and saving this new model.

For Top-K layer fine-tuning code in Appendix (xvi), we called upon and froze our codeT5-Base model and determined K, or the number of layers to unfreeze. This code block is where we altered K during our hyperparameter tuning process, altering 2 to 8 unfrozen layers. Preprocessing in Appendix (xvii) prepares the model for the directionality of translation (ie. C# to Java) and tokenizes the labels and model inputs. Finally, Appendix (xviii) defines our model parameters before initializing our training process. The code snippet there allowed us to alter the learning rate during our hyperparameter tuning process, ultimately deciding on a rate of 1e-4. Lastly, Appendix (xix) allows us to translate our test dataset in batches and retrieve the exact match and BLEU scores when compared to the ground truth labels.

The code snippets included in the Appendix are our main written code, and the code file submitted will have variations to this main code that are equivalent in general functionality. For instance, we included code that helps with batch translation tasks after our model trains, but did not include almost identical code, which only translates one instance.

Timeline:

Please refer to Appendix (xx) for the timeline table.

Research Log:

Originally, our group planned to implement from scratch a small transformer model and both train and fine-tune it specifically for code translation. After speaking with our liaison and reading more research papers, we realized that, given the time and computing resources, it would be very difficult to do all this from scratch and get decent results. Thus, our objective shifted to get a relatively small model to perform well on code translation tasks using parameter-efficient fine-tuning. This led us to pivot to a pre-trained model, where we chose the CodeT5 model with 220M parameters. We faced difficulties in finding good data for fine-tuning and evaluation. For OCR, we had to create our own dataset of code images with the corresponding code. Given the time constraint, we weren't able to construct extensive C# code data for the OCR dataset and had to rely on the evaluation performance of Python code, which was one of our initial languages. For CodeT5, we first stumbled upon the AVATAR dataset, which had 1-1 code translation between Java and Python. The paper mentioned how this supervised learning approach and data improved translation performance, but accessing this dataset was all but straightforward, hence, we pivoted to the code_x_glue dataset.

Conclusion:

Our projects sought to implement and evaluate the multi-modal pipeline that combines vision-based code identification and language translation. Specifically, our two-stage pipeline takes inputs of code screenshots of a source language and outputs an equivalent translation in a target language, especially emphasizing on Java and C#. Our task combined the OCR capabilities of Google's Vision API with a fine-tuned CodeT5 model using two different parameter-efficient approaches. We discovered that image preprocessing methods like ImageEnhance improved recognition, with NLD scores of 0.0339 and 0.0340 for Python and Java, respectively. For translation, our application of LoRA and Top-K fine-tuning on codet5-base achieved BLEU scores of up to 0.9064 and 0.9081 respectively, surpassing the results presented by Wang, Yue, et al. While our end-to-end pipeline that combines both submodels had reduced EM and BLEU scores, we attribute the deviation to recognition errors being exacerbated further down the pipeline. Ultimately, our pipeline still produces comparable results using parameter-efficient fine-tuning approaches, confirming our initial hypotheses on the effectiveness of such applications in the code translation domain. Some considerations for future work may be to compare our results with other, possibly more generalized, multimodal transformer models like OpenAI's GPT-4o that can complete similar translation tasks for baseline comparisons. Furthermore, we ideally want our model to incorporate more programming languages and explore other parameter-efficient fine-tuning methods. With more computation resources, we would've wanted to evaluate our approach on the full CodeT5 model.

References:

- Bao, Lingfeng, et al. *p2c: Denoising Code Extraction from Programming Screencasts*. ACM Transactions on Software Engineering and Methodology, vol. 29, no. 3, Association for Computing Machinery (ACM), June 2020, pp. 1–38. <https://doi.org/10.1145/3392093>.
- Khormi, Abdulkarim, Mohammad Alahmadi, and Sonia Haiduc. *A Study on the Accuracy of OCR Engines for Source Code Transcription from Programming Screencasts*. Proceedings of the 17th International Conference on Mining Software Repositories (MSR '20), ACM, 2020, pp. 1–11. <https://doi.org/10.1145/3379597.3387468>.
- Roziere, Baptiste, et al. *Unsupervised Translation of Programming Languages*. Proceedings of the 34th Conference on Neural Information Processing Systems (NeurIPS 2020), 2020, <https://arxiv.org/abs/2006.03511>.
- Liu, Fang, Jia Li, and Li Zhang. *Syntax and Domain Aware Model for Unsupervised Program Translation*. arXiv preprint arXiv:2302.03908v2, 2023. <https://arxiv.org/abs/2302.03908>.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen. *LoRA: Low-Rank Adaptation of Large Language Models*. arXiv 2021. <https://arxiv.org/pdf/2106.09685>.
- Houlsby, Neil, et al. "Parameter-Efficient Transfer Learning for NLP." *arXiv*, 13 June 2019, <https://arxiv.org/abs/1902.00751>.
- Wang, Yue, et al. "CodeT5: Identifier-aware Unified Pre-trained Encoder-Decoder Models for Code Understanding and Generation." *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, Association for Computational Linguistics, 2021. <https://arxiv.org/pdf/2109.00859v1>.
- Ahmad, Wasi Uddin, et al. "AVATAR: A Parallel Corpus for Java-Python Program Translation." *Findings of the Association for Computational Linguistics: ACL 2023*, Association for Computational Linguistics, 2023. <https://arxiv.org/pdf/2108.11590>

Datasets Reference:

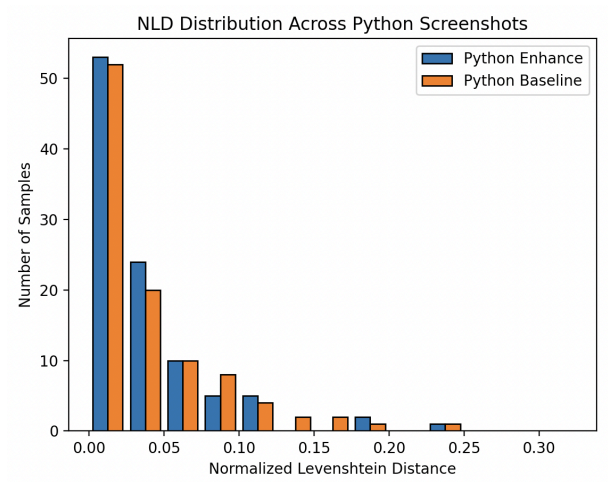
https://huggingface.co/datasets/google/code_x_glue_cc_code_to_code_trans

Appendix:

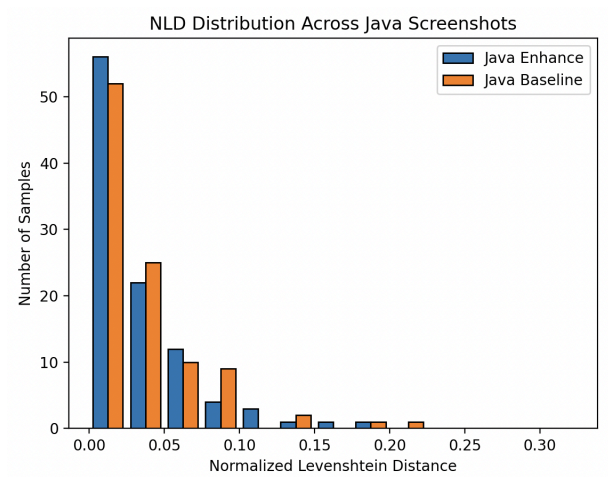
(i)

Language Dataset	Baseline NLD Score	Binarize NLD Score	Pillow NLD Score
Python	0.0390	0.0494	0.0339
Java	0.0364	0.0620	0.0340

(ii)



(iii)



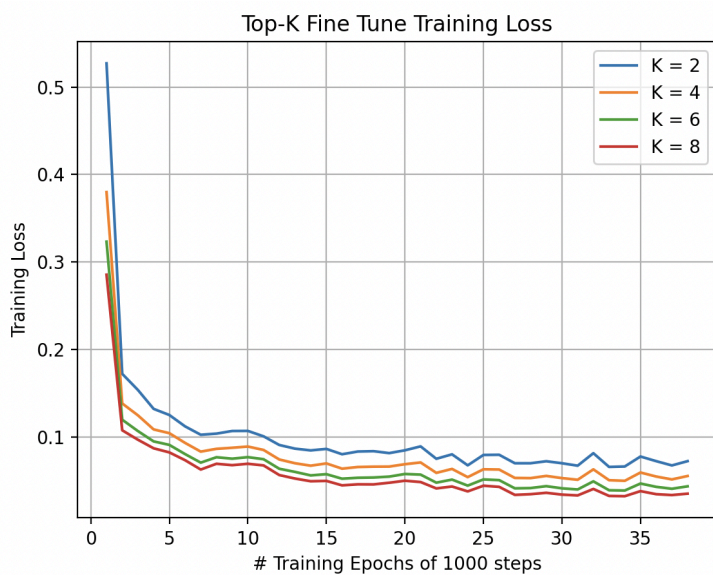
(iv)

	Java $\rightarrow$ C#		C# $\rightarrow$ Java	
Model	BLEU	Exact Match	BLEU	Exact Match
codet5-small	0.0007	0.0 %	0.0003	0.0 %
codet5-base	0.0161	0.0 %	0.0383	0.0 %

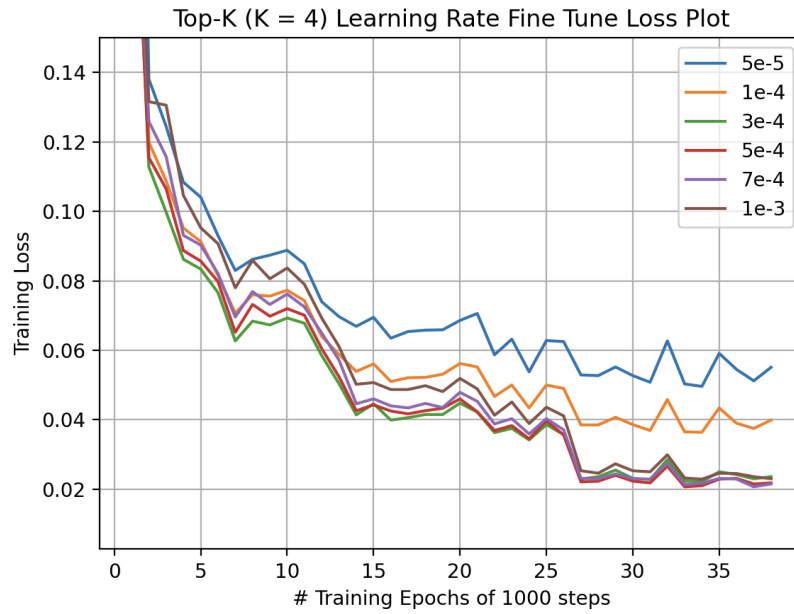
(v)

	Java $\rightarrow$ C#		C# $\rightarrow$ Java	
Model	BLEU	Exact Match	BLEU	Exact Match
codet5-base (LoRA)	0.9064	61.80%	0.8989	63.90%
codet5-base (Default FFT)	0.8403	65.90%	0.7987	66.90%

(vi)



(vii)



(viii)

	Java → C#		C# → Java	
Model	BLEU	Exact Match	BLEU	Exact Match
codet5-base (Top K = 2)	0.8404	51.50%	0.8517	55.70%
codet5-base (Top K = 4)	0.8658	54.30%	0.8723	59.30%
codet5-base (Top K = 6)	0.8846	56.40%	0.8824	60.80%
codet5-base (Top K = 8)	0.8924	57.70%	0.8917	62.10%
codet5-base (Top K = 8, lr = 5e-4)	0.9081	62.00%	0.8996	64.10%

(ix)

	Java → C#		C# → Java	
Model	BLEU	Exact Match	BLEU	Exact Match
Combined Model (LoRA)	0.7558	20.00%	0.6817	30.00%
Combined Model (Top-K)	0.7439	30.00%	0.6710	25.00%

(x)

```
def extract_code_from_image(image_path):
    os.environ["GOOGLE_APPLICATION_CREDENTIALS"] = '/content/ocr-for-gen-ai-fac0fbc83a73.json'
    client = vision.ImageAnnotatorClient()

    with io.open(image_path, 'rb') as image_file:
        content = image_file.read()

    image = vision.Image(content=content)
    response = client.document_text_detection(image=image)

    text = response.full_text_annotation.text
    return text
```

(xi)

```
def binarize_image(image_path):
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
    binary = cv2.adaptiveThreshold(img, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
                                   cv2.THRESH_BINARY, 11, 2)
    #cv2.imwrite("binarized.png", binary)
    return "binarized.png"

def preprocess_image(input_path, output_path="processed.png"):
    img = Image.open(input_path).convert("L") # grayscale
    enhancer = ImageEnhance.Contrast(img)
    enhanced_image = enhancer.enhance(2.0)
    enhanced_image.save(output_path)
    return output_path
```

(xii)

```
def normalized levenshtein(s1, s2):  
    dist = Levenshtein.distance(s1, s2)  
    max_len = max(len(s1), len(s2))  
  
    return dist / max_len if max_len > 0 else 0.0  
  
def log_nld(index, nld, log_file="log.txt"):   
    with open(log_file, "a") as f:  
        f.write(f"Image {index} | NLD: {nld:.4f}\n")  
  
def normalize(code):  
    return ''.join(code.strip().split())
```

(xiii)

```
def run(language,clean):  
    df = pd.read_csv('ocr_data.csv') # ground truth code from csv dataset  
  
    image_folder = f"/content/screenshots-gen-ai/{language}"  
  
    nld_iter = []  
    exact,accurate = 0,0  
  
    for i in range(100):  
        image_path = os.path.join(image_folder,f'{i}.png')  
        #print(f"Using image {i}: {image_path}")  
  
        if clean:  
            cleaned_image = preprocess_image(image_path) # preprocessing  
            #cleaned_image = binarize_image(image_path)  
            code_text = extract_code_from_image(cleaned_image) #code extraction  
        else:  
            code_text = extract_code_from_image(image_path)  
            cleaned_code = re.sub(r"\s+", "", code_text) #no whitespace  
  
        #groundTruth code from csv  
        if language == 'python':  
            code_gt = df.iloc[i]['Code'] # for python  
        elif language == 'java':  
            code_gt = df.iloc[100+i]['Code'] # for java  
  
        code_gt = code_gt.replace(r"\n", "\n")  
        code_groundTruth = re.sub(r"\s+", "", code_gt) #no whitespace  
  
        nld = normalized levenshtein(normalize(cleaned_code),normalize(code_groundTruth))  
        if nld >= 0.5:  
            print(f"Image {i}.png → NLD: {nld:.4f}")  
            print('code:',cleaned_code)  
            print('groundTruth:',code_groundTruth)  
        elif nld <= 0.05:  
            exact += 1  
            accurate += 1  
        elif nld <= 0.1:  
            accurate += 1  
        nld_iter.append(nld)  
        log_nld(i,nld=nld)  
  
    avg = sum(nld_iter)/len(nld_iter)  
    return nld_iter, avg, exact, accurate
```

(xiv)

```
import math
import torch.nn as nn
from transformers import (
    AutoModelForSeq2SeqLM,
    AutoTokenizer,
    TrainingArguments,
    Trainer,
    DataCollatorForSeq2Seq
)
from transformers.models.t5.modeling_t5 import T5Attention

class LoRALinear(nn.Linear):
    def __init__(self, in_features, out_features, bias=True, device=None,
                 dtype=None, lora_rank=8, lora_alpha=16, lora_dropout=0.1):
        super().__init__(in_features, out_features, bias=bias,
                         device=device, dtype=dtype)
        self.has_weights_merged = False
        if lora_rank > 0:
            self.lora_dropout = nn.Dropout(lora_dropout)
            self.lora_scaling = lora_alpha / lora_rank
            self.lora_A = nn.Parameter(torch.empty(lora_rank, in_features,
                                                    device=device, dtype=dtype))
            self.lora_B = nn.Parameter(torch.empty(out_features, lora_rank,
                                                    device=device, dtype=dtype))

            self.lora_A.requires_grad = True
            self.lora_B.requires_grad = True
            self.reset_parameters()

    def is_lora(self):
        return hasattr(self, "lora_A")

    def reset_parameters(self):
        nn.Linear.reset_parameters(self)
        if self.is_lora():
            nn.init.kaiming_uniform_(self.lora_A, a=math.sqrt(5))
            nn.init.zeros_(self.lora_B)

    def forward(self, input):
        output = super().forward(input)
        if self.is_lora() and not self.has_weights_merged:
            dropped = self.lora_dropout(input)
            A = dropped @ self.lora_A.T
            BA = A @ self.lora_B.T
            output += self.lora_scaling * BA
        return output

    def train(self, mode=True):
        super().train(mode)
        if not self.is_lora():
            return self
        if mode:
            if self.has_weights_merged:
                self.weight.data -= self.lora_scaling * (self.lora_B @ self.lora_A)
                self.has_weights_merged = False
            else:
                if not self.has_weights_merged:
                    self.weight.data += self.lora_scaling * (self.lora_B @ self.lora_A)
                    self.has_weights_merged = True
        return self

    def eval(self):
        super().eval()
        if self.is_lora() and not self.has_weights_merged:
            self.weight.data += self.lora_scaling * (self.lora_B @ self.lora_A)
            self.has_weights_merged = True
        return self
```

(xv)

```
def replace_with_lora(model, rank=8, alpha=16, target=("q","v")):
    for module in model.modules():
        if isinstance(module, T5Attention):
            for component in target:
                lin = getattr(module, component)
                lora_lin = LoRALinear(lin.in_features, lin.out_features, bias=False,
                                     lora_rank=rank, lora_alpha=alpha,
                                     lora_dropout=0.1)
                lora_lin.weight.data = lin.weight.data.clone()
                setattr(module, component, lora_lin)

def freeze_non_lora(model):
    for name, parameter in model.named_parameters():
        if "lora_A" in name or "lora_B" in name:
            parameter.requires_grad = True

tokenizer = AutoTokenizer.from_pretrained("Salesforce/codet5-base")

def encode(datapoint):
    # Java to CSharp
    model_inputs = tokenizer(datapoint["java"],truncation=True, max_length=512)
    labels = tokenizer(datapoint["cs"],truncation=True, max_length=512).input_ids

    # CSharp to Java
    model_inputs = tokenizer(datapoint["cs"],truncation=True, max_length=512)
    labels = tokenizer(datapoint["java"],truncation=True, max_length=512).input_ids
    model_inputs["labels"] = labels
    return model_inputs

train_ds = dataset['train'].map(encode, batched=True, remove_columns=dataset['train'].column_names)
```

(xvi)

```
model_name = "Salesforce/codet5-base"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)

# layer freeze
for param in model.parameters():
    param.requires_grad = False

K = 8 # adjust for number of layers

# layer unfreeze
for layer in model.encoder.block[-K:]:
    for param in layer.parameters():
        param.requires_grad = True
for layer in model.decoder.block[-K:]:
    for param in layer.parameters():
        param.requires_grad = True
for param in model.shared.parameters():
    param.requires_grad = True
for param in model.lm_head.parameters():
    param.requires_grad = True

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
```

(xvii)

```
def preprocess(example):
    # for Java to C#
    #inputs = ["translate java to CSharp: " + src for src in example["java"]]
    #targets = example["cs"]

    # for C# to Java
    inputs = ["translate CSharp to java: " + src for src in example["cs"]]
    targets = example["java"]

    model_inputs = tokenizer(
        inputs,
        truncation=True,
        padding="max_length",
        max_length=256
    )

    labels = tokenizer(
        targets,
        truncation=True,
        padding="max_length",
        max_length=256
    ).input_ids

    model_inputs["labels"] = labels
    return model_inputs
```

```
tokenized_dataset = dataset.map(preprocess, batched=True)
```

(xviii)

```
### Top K training
from transformers import Seq2SeqTrainer, Seq2SeqTrainingArguments, DataCollatorForSeq2Seq

training_args = Seq2SeqTrainingArguments(
    output_dir=f"./codet5-topk-ft{K}",
    run_name=f"./codet5-topk-ft",
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    learning_rate=5e-4, #original 5e-5
    num_train_epochs=3,
    logging_dir="./logs",
    fp16=True,
    save_steps=500,
    logging_steps=100,
    eval_steps=500,
    save_total_limit=2
)

trainer = Seq2SeqTrainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset["train"],
    eval_dataset=tokenized_dataset["validation"],
    tokenizer=tokenizer,
    data_collator=DataCollatorForSeq2Seq(tokenizer, model=model),
)

trainer.train()
trainer.evaluate()
```

(xix)

```
def batch_translate_examples(examples):
    #Java to CSharp
    inputs = ["translate java to CSharp: " + src for src in examples["java"]]

    #CSharp to Java
    #inputs = ["translate CSharp to java: " + src for src in examples["cs"]]

    tokenized_inputs = tokenizer(
        inputs,
        return_tensors="pt",
        padding="max_length",
        truncation=True,
        max_length=512
    ).to(device)

    # predictions
    outputs = model.generate(
        **tokenized_inputs,
        max_length=512,
        num_beams=5,
    )

    # outputs to strings
    predictions = tokenizer.batch_decode(outputs, skip_special_tokens=True)
    return {"predicted": predictions}
```

```
translated_test_dataset = dataset["test"].map(batch_translate_examples, batched=True, batch_size=8)
```

[Show hidden output](#)

```
preds = translated_test_dataset["predicted"]
refs = translated_test_dataset["java"]

exact_matches = sum(p.strip() == r.strip() for p, r in zip(preds, refs))
em_score = exact_matches / len(preds) * 100

print(f"Exact Match: {em_score:.2f}%")

bleu = evaluate.load("bleu")
wrapped_refs = [[ref] for ref in refs]
bleu_score = bleu.compute(predictions=preds, references=wrapped_refs)
print("BLEU:", bleu_score["bleu"])
```